

Scenario 1: Pretrained Model Predicts Only One Class in Object Detection

Context: You fine-tuned a pretrained YOLOv5 model on a custom dataset (detecting helmets). After training, it always predicts only one class.

Problem: Despite having multiple classes in the dataset, the model outputs only "helmet" for every object

Logical Solution:

Fixes:

1. Check class config: Verify `data.yaml` has correct number of classes (`nc`) and names list order matches your labels.
2. Fix labels: Make sure annotations aren't all tagged as "helmet" by mistake.
3. Balance data: Add more examples of other classes (no helmet, person, etc.).
4. Retrain properly: Use same `data.yaml` for training and testing, watch per-class accuracy.
5. Test confidence: Lower detection threshold to see if other classes appear weakly.

Scenario 2: Transfer Learning Model Trains Too Slowly on Small Dataset

Context: You are using a pretrained ResNet50 model for face mask classification (2 classes). You fine-tune the entire model on a small dataset (500 images).

Problem: Training takes too long and doesn't converge well.

Logical Solution:

Fixes:

1. Freeze base layers: Set `base\_model.trainable=False` for early conv layers.
2. Train classifier only: Only update final Dense layers first
3. Smaller batch size: Use batch_size=16/32 for small dataset stability.
4. Lower learning rate: lr=1e-4 (not 1e-3) prevents destroying pretrained weights.

Scenario 3: Transfer Learning Works on Training Data but Fails on New Environment

Context: You used a pretrained YOLOv8 model trained on outdoor images to detect tigers. But when used in night-mode CCTV footage, detection fails.

Problem: Domain shift between training data (bright daylight) and real-world application (night).

Logical Solution:

Fixes:

1. Add night images: Mix dark CCTV tiger videos into training data
2. Darkness augmentation: Train with random brightness drops (0.1-0.8)
3. Separate night model: Fine-tune copy of YOLOv8 just on night

Scenario 4: Object Detection Model Shows Bounding Boxes but Misses Object

Context: You trained a YOLOv5 model using transfer learning on a small vehicle dataset. Boxes are drawn, but not aligned with objects.

Problem: Bounding box coordinates are incorrect due to **wrong label format or image resizing** mismatch.

Logical Solution:

Fixes:

1. Check label format: Use center_x, center_y, width, height (normalized 0-1)
2. Verify image sizes: Labels must match exact training image dimensions
3. Fix resize mismatch: Use same resize (640x640) for training and inference
4. Re-annotate: Manually check 10 labels with `labellmg` → YOLO format

Scenario 5: Model Gives High mAP on Training But Low Real-Time Accuracy

Context: You used pretrained YOLOv8 for multi-class object detection (helmet, vest, mask). mAP on test set is 90%, but real-time inference on webcam is inaccurate.

Problem: Difference in resolution, camera angle, or image quality between webcam and training data.
Poor generalization due to **dataset bias**.

Logical Solution:

Fixes:

1. Match webcam input: Resize frames to training size (640x640)
2. Add webcam data: Record 100+ webcam clips → label and retrain
3. Realistic augmentation: Blur, brightness changes, camera shake

Scenario 6: VGG16 Model Very Slow During Training

Project: Fashion product classification using VGG16

Problem: Model takes too long to train even with a small dataset and batch size.

Logical Solution:

Fixes:

1. Freeze base: base_model.trainable=False (train classifier only)
2. Smaller batch: batch_size=16 (fits GPU memory)
3. Reduce image size: 224x224 → 128x128 (4x faster)

Scenario 7: InceptionV3 Fails to Train Due to Input Size Mismatch

Project: Dog breed classification using **InceptionV3**

Problem: Runtime error due to input size mismatch.

Logical Solution:

Fixes:

1. Resize input: (img_height, img_width) = (299, 299)
2. Check preprocessing: Use preprocess_input from keras.applications.inception_v3
3. Verify batch shape: (batch_size, 299, 299, 3) before model.fit()